

Purpose

The system provides a platform for organizations to understand stakeholders' expectations about issues of organizational interests. In a regular business setting, for instance, to measure the engineer team's sentiments on the time it will take to complete a given feature, organization leaders request information from managers directly responsible for the projects, creating a chain of information transmission that risks the information loss at different layers. However, our platform enables the leaders to create events such as the status of a feature, which contains markets that the team can directly invest in. Selected members in the organization are invited and incentivized to make the most likely prediction. This eliminates social desirability bias and ensures honesty. It also removes the need for intermediary reports and enables transparent and data-driven analysis along with real-time events in the organization (e.g. an engineer leaving the team).

Specifically, our system allows organization leaders to create events that are open only to users of certain roles within the organization and are given tokens to make the investments. In this exchange betting system, an individual places a back bet in a binary market at a token cost between 0 and 1. If another individual is willing to place a lay bet at the same odds such that the cost of the two bets adds up to 1, they will be matched, and the winner gets the amount, which is 1 token.

However, the purpose of the platform is not simply outcome investment, rather, it is the sentiment data that come with it. So, our system also allows for specific people and roles within the organization to have viewing capability of the market predictions, along with the outcome of various data analysis functions to unveil hidden traits of the markets.

Functionality

The system is organized hierarchically. A user creates an organization, an organization can create events, and markets are created within an event. Within an organization, the founding user is the organization leader that is responsible for the event creation and constraints; an organization leader can also create markets within events and get the exclusive access to the more detailed analysis of the market. Users can also join the organization under a specific role, where they may be designated a bettor (can invest in binary outcome markets), analyst (can view the current outcome percentage along with certain more specific analytical functions) or viewer (can view the current outcome percentage) of certain market depending on their role. If a user wishes to create markets, the organization leader designate the user specifically as a market maker under a specific event.

For the system to function properly, it needs to authenticate and designate access based on user roles or user ids. All functionalities should work, such as creating events and placing bets. Market and event constraints cannot be violated. Once an organization leader ends a market, tokens need to be correctly distributed to user accounts. The sum of all tokens held by users plus tokens in the market must exactly equal the initial token supply. The system needs to be stressed tested by having multiple bets made or markets created simultaneously while maintaining a good performance. Moreover, when one server shuts down, the system needs to distribute workload and data at the failed server to the other servers in the system.

Challenges

1. Atomic Transactions

If user A has 1 token and tries to place two different bets on two different markets at the exact same time, a distributed system might allow both if the database is partitioned. Therefore, the system must act as if there is only one single copy of the data, even if it's spread across multiple servers.

2. Transaction Order

If user A offers a bet at 0.5 and user B offers 0.4, the person who gets matched first is the one who submitted their request first. In a distributed system, clock skew can make it hard to tell who was actually first. This system uses Kafka to provide a total order of events so that every node agrees on the sequence of bets.

3. Idempotent Payout

When a Leader closes a market, the system needs to distribute tokens to multiple users. To ensure every user is paid once even if the server crashes during the payout process, the payout needs to be idempotent so that at a network retry, the winners are not paid twice. The state transition from Escrow to User Wallet must be an atomic operation across the distributed cluster.

4. Availability vs. Consistency (The CAP Theorem)

For a betting system, consistency is essential. The system needs to stop accepting bets if there is a network issue instead of risking recording incorrect balances.

5. Eventual Consistency for Analysis

While the betting must be sequentially consistent, the Analysis view for leaders can be eventually consistent. Betting happens on a sequentially consistent write database, and the Leader's real-time charts use a read replica that is slower.

6. Database Locking

When a match occurs, three records are modified simultaneously: User A's balance, User B's balance, and the market tokens. If there is a lot of traffic, traditional database locking can create a bottleneck and slow the system. If we need to scale up the system, we can implement concurrency control and retry logic.

7. Latency

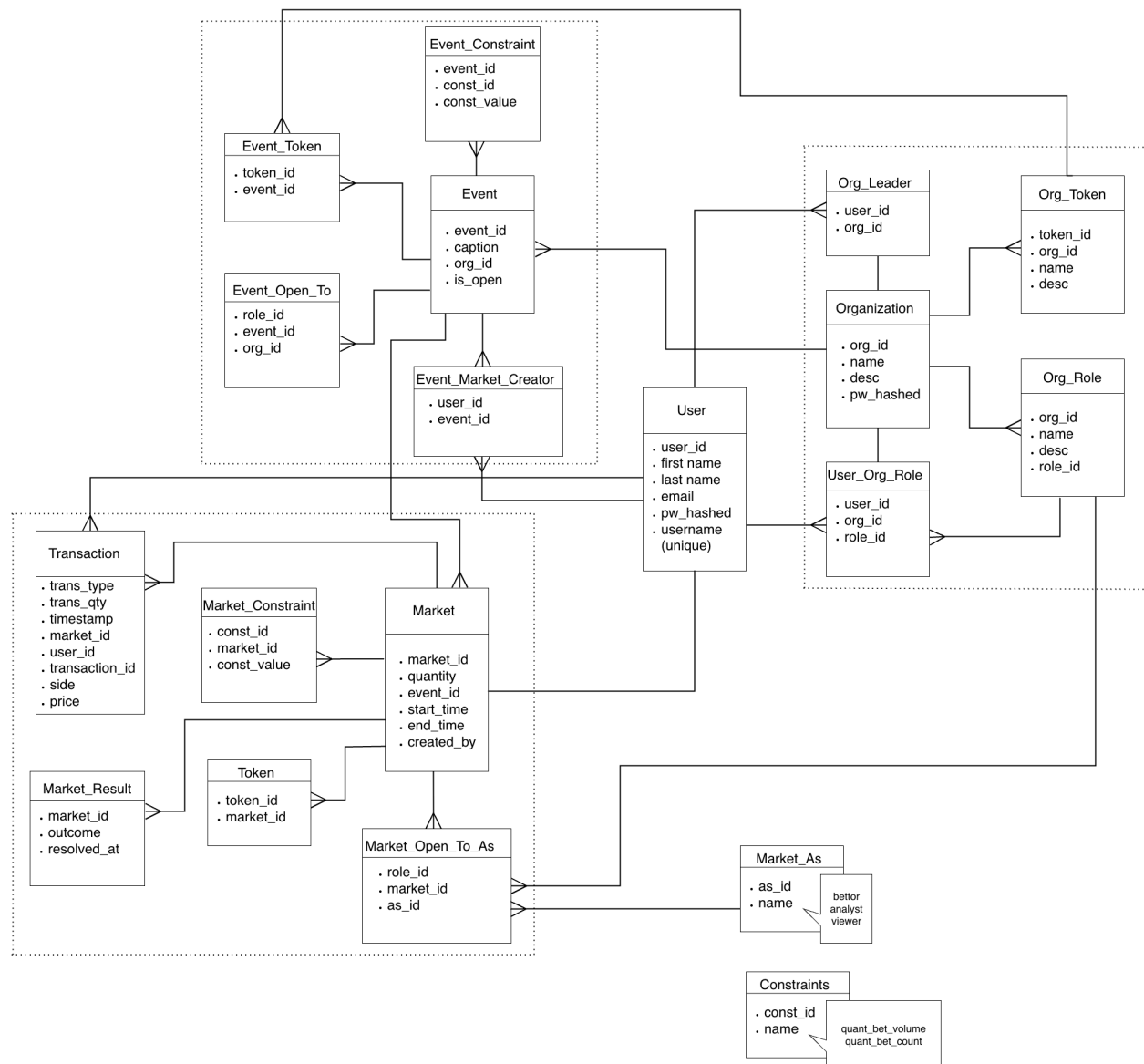
The application theoretically will handle a large volume of betting for specific markets, making throughput a top priority in the long-term. However, at all levels of its implementation, to provide a live data platform of business sentiments, rapid communication with the server for quick updates is important.

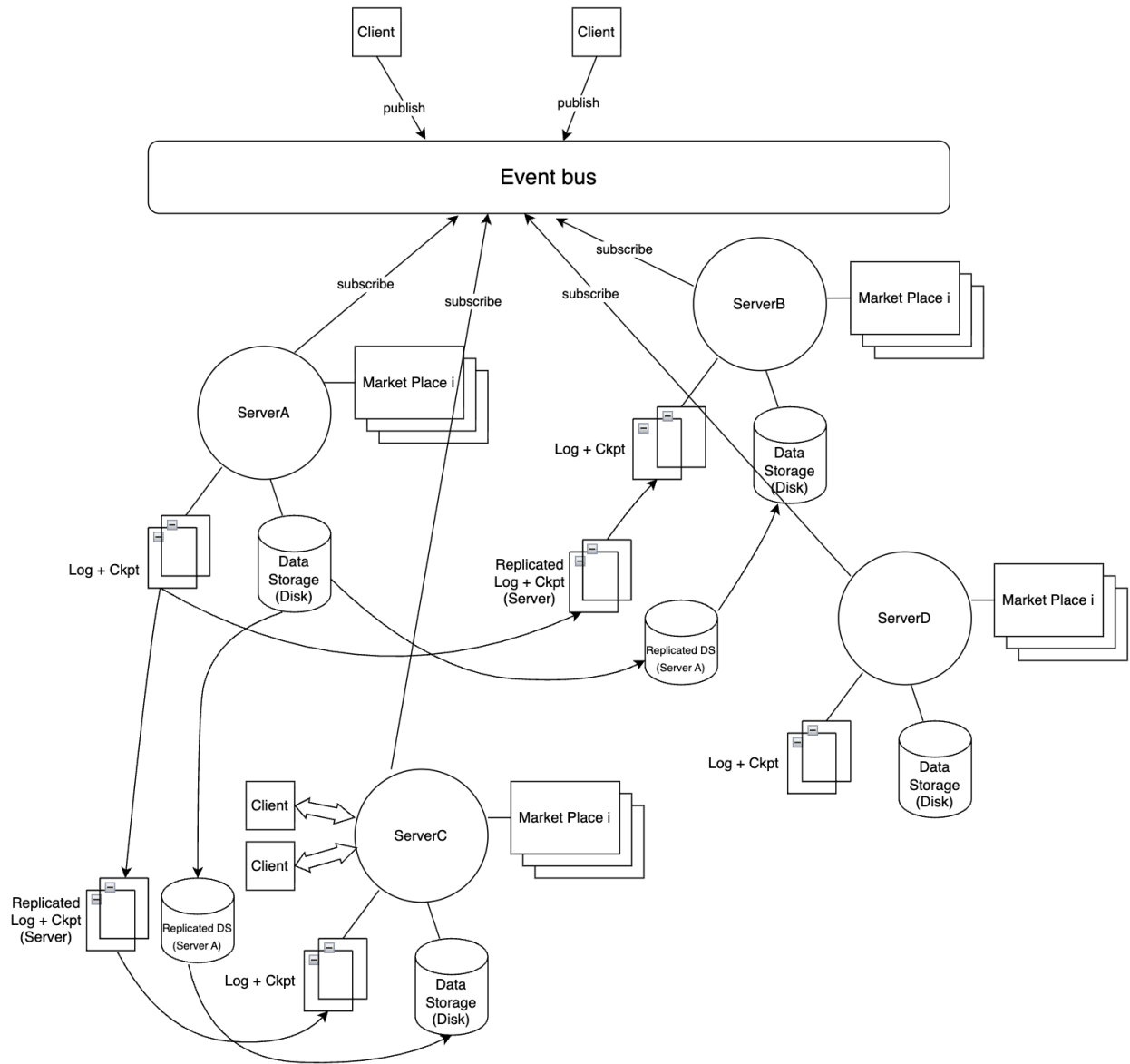
Architecture

Users whose `user_id` corresponds to `org_id` in `org_leader` table can create events under the organization. An organization can coin different types of tokens that can be used in different markets/events to prevent tokens designated for an event to be used in other events. An organization also creates roles for the organization (engineer, marketer, businesss, etc) in the `org_role` table, so when users join via the passcode, they join with a specific role attached to their name via the `user_org_role` table. During the creation of an event, an event record is generated with entries from the `event_constraints` table (e.g., `quant_bet_volume`) selected by the leader and added to the `event_constraint` table. Similarly, when a market is created under that event, it is given a set of constraints through `market_constraint`.

A user's relationship to a market is defined by joining `user_org_role`, specified by their `user_id`, with `market_open_to_as`. To be a bettor for a market, the app checks if the `user_id` has a

role_id that matches the as_id of the role “bettor” for that specific market_id. If the user passes this check, they can invest and a transaction record is created with a trans_type (e.g., "bet placement"), and checked against the market_constraint. The backend scans the Transaction table for an opposing side where price_existing + price_new = 1. If matched, the token is moved to a system-controlled user_id.





The system uses a relational identification framework with one-to-one, one-to-many and many-to-many relationships between different database items as shown in the graph above. Database entities (e.g. users, events) have primary keys (id) that are UUIDs (Universally Unique Identifiers). Tables like Organization, org_role, and market_as have name, description or caption fields that are for display instead of internal logic. The user table explicitly enforces username to

be unique. The system uses hierarchical discovery to lookup items. For instance, to find a market, the system queries user → user_org_role → organization → event → market.

When individual nodes fail, the event bus can notice it and publish information so that the failed node's neighboring servers that are subscribed to the topic notice it and take on the loads of the failed server. To ensure a failed server's data is preserved, the data is replicated across servers every time a new transaction is made or a market is created. If an operation at one server fails, a user can retry the operation until it executes successfully without risking duplicate execution due to idempotency.

Progress

The functionality of the system on a higher level is largely complete. We decided to fully build out the core features of the application first, rather than the abstract and distributed processes that empower them because we felt it most important to define the ideal outcome of the application before the low-level optimization engineering. This way, we could construct specific communication and data access tricks to optimize for our structure specifically.

- i. Database

```

mysql> describe organization;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| org_id | int | NO | PRI | NULL | auto_increment |
| name | varchar(255) | NO | | NULL | |
| description | text | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> describe organization_role;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| org_id | int | NO | PRI | NULL | |
| role | varchar(100) | NO | PRI | NULL | |
| description | text | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> describe organization_leader;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| org_id | int | NO | PRI | NULL | |
| user_id | int | NO | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> describe organization_token
-> ;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| token_id | int | NO | PRI | NULL | auto_increment |
| org_id | int | NO | MUL | NULL | |
| name | varchar(255) | NO | | NULL | |
| description | text | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Currently, running on the Notre Dame db8 MySQL servers under database tmitch23, is each object table necessary for the application with their specific attributes defined. More specifically, each object currently specifies the necessary datatypes, sizes, and other important features per attribute. The objects also have automated ids, along with their referenced foreign keys to objects they contain many-to-one, one-to-many, and one-to-one relationships with. To remove duplicate data and ensure maximal data integrity, we constructed it with the 3 normal forms, avoiding any non-atomic attributes and producing objects with full primary key dependency without transitive dependency. This was especially challenging for some of the more specific objects like user_org_role, where the one-to-many relationship between the user and the object defines the specific role that a user has within an organization. Creating this object came after trial-and-error, attempting to register users within the org_role object, however, this created a

partial dependency between user_id and org_id, which violated the 2nd normal form. Therefore, we broke this up into a new object, user_org_role.

ii. Backend Utility Functions

```
def create_o_role(cursor, db, user_id, organization_id, name, desc):  
    # Check if user is the organization leader  
    cursor.execute(  
        """  
        SELECT 1  
        FROM organization_leader  
        WHERE org_id = %s AND user_id = %s  
        """,  
        (organization_id, user_id),  
    )  
    if cursor.fetchone() is None:  
        return None  
  
    # Check if role with the same name already exists in the organization  
    cursor.execute(  
        """  
        SELECT 1  
        FROM organization_role  
        WHERE org_id = %s AND role = %s  
        """,  
        (organization_id, name),  
    )  
    if cursor.fetchone() is not None:  
        return None  
  
    # Create role and insert it into the database, returning the role ID  
    cursor.execute(  
        """  
        INSERT INTO organization_role (org_id, role, description)  
        VALUES (%s, %s, %s)  
        """,  
        (organization_id, name, desc),  
    )  
    db.commit()  
  
    return name
```

With the database installed, it was necessary to create the methods that put it to use, including the action methods for writing new data in and the passive methods for reading data from the marketplace. These methods allow for the next task at hand, network communication, to be done at a much faster pace, not having to worry about the specific methods the RPC functions request to use, rather just calling them from a different file, similar to the way we did it in our class project.

The process for making the action methods was more than insertions and deletions. It also entailed multiple authentication, and duplication checks to ensure that no function call could obstruct the functionality of the application. For example, it was described in the earlier sections that only specific users can produce events and markets and their rules under an organization. So, when a user calls any of the backend functions for these creations, the function authenticates whether or not the user has access to the function. Then, to avoid duplications, the function checks if the same data already exists within the organization. Both processes must be done through SQL queries, which were pre-written before the backend functions were produced. Then, once all checks pass, we insert the data directly into the database.

An important asset that has not been fully implemented for the backend functions is idempotency, although it has been largely. Currently, running checks on the existence of already present tokens, constraints, and market makers enforces a level of idempotency, but other functions exist, such as those for transactions still need working. We are planning to integrate a idempotency key to make sure we avoid any duplicated transactions for market investment.

The process for making the passive methods was a two-step process for each function. A passive method once again entails authenticating a user's access to a function through SQL queries, but the end goal of the process is also a SQL query where the user gets more data on a market by calling more specific data functions. Here, we also pre-produced SQL functions that found specific data on the market including the total volume of the market, and the prediction percentage of the market if it included only whale bets or only bets spanning two timestamps.